

Relatório de Decisões de Implementação

1. Introdução

Esta etapa consolidou a modelagem do sistema hospitalar em uma arquitetura que combina stored procedures, triggers e um ORM (SQLAlchemy), equilibrando performance, segurança e facilidade de manutenção sem comprometer a coerência do modelo relacional já validado na Etapa 1.

2. Triggers vs. Procedures

A escolha entre as duas seguiu um critério prático: procedures foram empregadas para operações executadas por demanda e com fluxo transacional bem definido (como o registro de um atendimento completo, que envolve múltiplas tabelas e precisa ser atômico), enquanto triggers foram reservados para validações e efeitos colaterais automáticos, disparados diretamente por eventos do banco (INSERT/UPDATE/DELETE), como a auditoria de atendimentos e a verificação de sobreposição de escala.

Essa separação torna o fluxo mais previsível: a aplicação decide quando chamar uma procedure, mas não precisa se lembrar de acionar uma trigger, já que ela é garantida pelo próprio SGBD, independentemente de quem alterou os dados. Ainda assim, triggers foram usados com moderação (reservados às regras cuja automação trazia valor real, como auditoria e integridade), já que aumentam a complexidade de depuração ao disparar efeitos indiretos nem sempre visíveis no código da aplicação.

3. Views

As três views implementadas (vw_pacientes_internados, vw_residentes_sem_supervisor e vw_estatisticas_atendimentos_mensal) não são tabelas temporárias: são consultas nomeadas e armazenadas no catálogo do banco, que não guardam dados por si mesmas (cada acesso a uma view executa novamente a lógica subjacente contra as tabelas reais). Por isso, o ganho principal não é velocidade bruta (esse seria o papel de uma materialized view, que armazena fisicamente o resultado e precisa ser atualizada periodicamente), e sim organização: consultas mais elaboradas, com múltiplos joins e agregações (como listar pacientes atualmente internados ou calcular estatísticas mensais de atendimento por unidade) ficam encapsuladas atrás de um nome simples, reaproveitável tanto pela aplicação Flask quanto por ferramentas externas de análise, sem repetir a mesma lógica de consulta em vários pontos do código.

4. Escolha da ORM

O SQLAlchemy foi escolhido para integrar o banco a uma aplicação Flask de forma organizada, mapeando tabelas e relacionamentos em classes Python e reduzindo a necessidade de SQL manual nas operações de CRUD (cadastro, listagem, edição e detalhamento de pacientes, profissionais e atendimentos). Para operações mais sofisticadas, como o cadastro de um atendimento completo, a aplicação combina o ORM com procedures SQL: o ORM cuida do fluxo geral da aplicação, e o banco garante a atomicidade da transação.

5. Consultas Avançadas, Carregamento e Concorrência

As três consultas avançadas exigidas (preceptores de residentes que atenderam pacientes flamenguistas, último atendimento por paciente e percentual de procedimentos de alto risco por residente) foram implementadas com a DSL do SQLAlchemy (join, filter, group_by e funções de agregação), sem recorrer a SQL cru. O carregamento de relacionamentos usa lazy loading, a estratégia padrão do SQLAlchemy, na qual os dados relacionados só são buscados quando efetivamente acessados; o carregamento adiantado (eager loading, via joinedload/selectinload) fica

registrado como próximo passo de otimização para consultas com listas grandes e relacionamentos aninhados.

Para simular duas transações concorrentes escalando o mesmo residente no mesmo dia, turno e unidade, foi usado um lock pessimista via `with_for_update()` sobre o registro do residente, forçando a segunda transação a aguardar a conclusão da primeira antes de verificar o conflito, eliminando a janela de corrida entre as duas. Um lock pessimista (em vez de um controle otimista baseado em coluna de versão) se justifica porque o problema é impedir a criação duplicada de um registro que ainda não existe, e não detectar um conflito de edição sobre um registro já existente.

6. Considerações Finais

A implementação busca equilíbrio entre desempenho, segurança e clareza: procedures para processos transacionais encapsulados, triggers apenas onde a automação é imprescindível, e o SQLAlchemy para simplificar a camada de aplicação (mantendo o sistema modular, mais fácil de manter e coerente com os objetivos desta etapa).